OPERATIONS WORKBOOK

Timed Coding and System Design Rehearsal Workbook

Two controlled, interview-shaped practice blocks with protected decision clocks, completed evidence records, and practical debriefs.

Use time pressure to test decision order while protecting clarification, invariants, testing, trade-offs, and failure handling.

VERSION 1.1

FORMAT GUIDE + WORKED EXAMPLES

STATUS LEARNER READY

OPERATIONS / TIMED-TECHNICAL-REHEARSAL

REHEARSE THE DECISION ORDER

Use the clock without racing

A timed rehearsal tests whether clarification, reasoning, testing, and trade-offs still appear when the interview window is fixed.

CORE IDEA

The timer measures decision order, not intelligence or readiness. Protect the steps that make an answer trustworthy; compress detail before silently skipping the contract, evidence, testing, or failure handling.

01	02	03	04
Set	**Run**	**Capture**	**Correct**
Choose one coding prompt and one design prompt, then lock time, tools, and assistance.	Speak decisions aloud and mark the moments where the answer changes direction.	Save the prompt, artifacts, score, quote, timestamp, and first missed decision.	Choose one behavior to test next; do not turn the debrief into a new syllabus.

SELF-REVIEW

Solo mode

Record audio or screen only when permitted. Use visible timestamps and score from artifacts immediately after the run.

PAIRED REVIEW

Observer mode

Ask the observer to note quotes and decision moments. Neutral prompts may clarify; they must not rescue the solution.

The 80-minute practice block

- [ ] Coding rehearsal: 30-minute attempt plus 10-minute evidence capture.
- [ ] System-design rehearsal: 35-minute attempt plus 5-minute evidence capture.
- [ ] Use the stated clock, the task-specific rubric, one 0 / 1 / 2 evidence scale, and the same assistance policy for each run.
- [ ] Save the required artifacts before choosing a correction.

PREPARE THE CONDITIONS

Select prompts that expose decisions

Use an active Library mock or a prompt you have not recently solved. The goal is retrieval and explanation, not recognition from memory.

Block	Prompt choice	Target dimension	Locked conditions
Coding	One sliding-window, graph, or dynamic-programming prompt	The decision you want the clock to expose	30 min; editor; language; no solution lookup
Design	One URL, notification, storage, or scheduling prompt	Requirements, scale, trade-off, or failure handling	35 min; blank canvas; calculator policy stated
Recording	Audio, screen, observer notes, or written timestamps	Exact quote and decision moment	Permission confirmed; no confidential prompt capture
Evidence	Code and tests; diagram and decision ledger	Observable result, not a feeling	Scored immediately with the same rubric

Illustrative setup record

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Field	Coding block	System-design block
Prompt / source	Fictional fresh prompt: longest substring containing at most two distinct characters.	Fictional fresh notification variant: preserve delivery state while one provider is slow.
Start time / end time	Example 09:00-09:30; evidence capture 09:30-09:40.	Example 09:45-10:20; evidence capture 10:20-10:25.
Tools / assistance	Local editor and tests; no lookup or hints; speak major decisions aloud.	Blank canvas and calculator; no component suggestions; one neutral failure probe.
Completion evidence	Code, eceba trace, edge tests, invariant timestamp, complexity, and one missed decision.	Diagram, three assumption-to-decision rows, failure timeline, metric, and one trade-off.

PROMPT RULE

Use the Library mock as a prompt source, not as an answer script. If you remember the solution path, choose another prompt and record why.

CODING BLOCK

Protect the contract, invariant, and tests

The 30-minute run keeps the roadmap task interview-shaped. The following 10 minutes preserve evidence before memory rewrites the attempt.

Clock	Decision to make	What the learner says or saves
0-5 min	Clarify contract, constraints, and examples	Inputs, outputs, edge rules, one normal example, one boundary example
5-10 min	Choose approach and state invariant	Pattern cue, state, invariant, rejected alternative, expected complexity
10-25 min	Implement with audible checkpoints	What each block preserves; any changed assumption; uncertainty named
25-30 min	Test and close	Empty/minimum, boundary or duplicate, main transition, time and space complexity
30-40 min	Capture evidence	Artifacts, scores, quote/timestamp, first missed edge, one correction

If you are stuck

01
Restate
Say the contract and the exact uncertainty.

02
Shrink
Use the smallest example that forces the main state change.

03
Baseline
Name a correct simple approach before optimizing.

04
Resume
Choose the next observable step and state its cost.

TIME-PRESSURE RULE

Do not hide. A visible correct baseline plus a named optimization gap is stronger evidence than silent guessing or an unexplained code jump.

WORKED CODING RECORD

Make a sliding-window solution inspectable

This completed example solves a fresh at-most-two-distinct-characters prompt. Use its contract, invariant, trace, and test reasoning as a standard for your own record, not an answer to memorize.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Artifact	Illustrative record
Contract and constraints	Input is a string; return longest contiguous length with at most two distinct characters. Empty returns 0; character model must be clarified.
Normal + boundary example	eceba -> 3 (ece); empty -> 0; aa -> 2; abaccc -> 4 (accc).
Pattern cue / approach	A right extension can create a third character; shrink left until at most two remain. Use a sliding window, not global frequency counts.
State and invariant	Counts represent only the current window. After shrinking, it contains at most two distinct characters; best is updated only then.
Rejected alternative + reason	Enumerating every substring is correct but quadratic or worse. Sorting destroys contiguity, so it changes the problem.
Time / space complexity	Both pointers move forward at most n times: O(n). With limit two, the count map has at most three keys during repair, so O(1) auxiliary space.
First missed edge + timestamp	At 18:00, eceba exposes update-before-shrink. At 26:00, abaccc passes after moving best update after the shrinking loop.

Score	Evidence standard
0 - absent	The behavior never appears or appears only after the run.
1 - prompted / late	The behavior appears after a rescue, late realization, or unexplained change.
2 - independent	The behavior appears at the correct decision point and is supported by an artifact.

WORKED CODING EVIDENCE

Score the attempt behind the code

These fictional timestamps refer to the same at-most-two-distinct attempt. Correct final code can still contain late reasoning; the evidence shows what should move earlier.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Dimension	Score + evidence	Quote, timestamp, code, or test evidence
Clarification / contract	2	01:00: distinguishes substring from subsequence and confirms at most two distinct characters.
Approach / decision order	2	04:20: expansion may break the constraint; shrinking repairs it without revisiting old starts.
State / invariant	1	18:00: invariant first stated after the update-before-shrink bug appears.
Correctness / recovery	1	20:30: moves best update after the while loop, then traces counts and boundaries.
Complexity / alternatives	2	24:00: each pointer moves at most n; sorting rejected because order matters.
Testing / communication	1	26:00: eceba and abaccc pass; empty and aa verified, but the first pointer repair was initially silent.

Worked evidence example

Observed moment	Cause hypothesis	Smallest correction
Invariant stated at minute 18 after implementation drifted	Planning named the pattern but not what the window must preserve	State the invariant before the first loop and re-check it at each pointer move
Final duplicate boundary test was missed	Testing started without a written edge list	Write four test labels during clarification; run them in the last five minutes

DO NOT COPY THE EXAMPLE

Use it to separate symptom, cause, and correction. Your correction must attach to your own evidence and decision cue.

SYSTEM-DESIGN BLOCK

Make assumptions reversible

A strong timed design answer connects requirements and scale to components, then closes with bottlenecks, recovery, and one deliberate trade-off.

Clock	Decision to make	What the learner says or saves
0-5 min	Requirements and non-goals	Users, core actions, latency/durability needs, boundaries, explicit exclusions
5-9 min	Order-of-magnitude scale	Traffic, storage, peaks, retention, and which estimate changes the design
9-17 min	API, data, and main flow	Contract, source of truth, write/read path, key entities, access pattern
17-25 min	Architecture and bottleneck	Components, ownership boundaries, first saturation or contention point
25-31 min	Failure and recovery	Failure mode, detection signal, containment, retry/idempotency or fallback
31-35 min	Trade-off and operations close	Alternative rejected, consequence accepted, observability, next question
35-40 min	Capture evidence	Diagram, three assumptions, decision ledger, score, quote, correction

COMPRESSION RULE

When time is short, reduce breadth and say what is deferred. Never silently skip requirements, a source of truth, the primary bottleneck, or failure handling.

WORKED DESIGN LEDGER

Connect assumptions to decisions and reversal points

The illustrative notification design below makes every assumption revisitable. In your next run, write three comparable rows in a notebook with your actual estimates and guarantees.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Assumption	Decision caused	Cost / trade-off	What would change it
Peak 5,000 events/s; average fan-out of two recipients is an explicit estimate.	Plan for 10,000 recipient deliveries/s before channel fan-out; separate durable event acceptance from delivery workers.	More queue and status state; capacity depends on real fan-out and provider quotas.	A hot tenant or fan-out of 100 changes partitioning, admission control, and capacity math.
Product accepts delayed delivery but must not lose accepted events.	Acknowledge after durable event write; retry delivery asynchronously with bounded backoff.	Status is eventually updated; backlog and replay ownership must be explicit.	A strict immediate-send promise requires a separate synchronous path and a different failure contract.
A provider timeout may hide a successful send.	Use stable delivery ID and provider idempotency where supported; otherwise expose uncertainty and reconcile.	Extra state and possible manual resolution; a database flag alone cannot prove external exactly-once effects.	Verified provider semantics or a product-approved duplicate policy changes the retry boundary.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Design evidence	Illustrative record
Source of truth + data flow	Durable event store -> recipient/channel work -> provider -> delivery-state record. A stable event/recipient/channel ID links retries.
Primary bottleneck + limit	Provider throughput quota, not queue size, bounds drain rate. At peak, compare arrival rate with measured service rate by channel.
Failure / recovery / metric	Provider outage: bounded retries, dead letters, controlled re-drive. Watch oldest-event age and failed-delivery rate, then validate recovery on a small batch.
Trade-off to revisit	Global FIFO is simpler but lets one tenant delay others; add fairness when per-tenant age or load evidence justifies it.

DECISION TEST

If an assumption changes, point to the component, data path, or operating policy that changes with it. If nothing changes, the assumption may not be decision-relevant.

WORKED DESIGN EVIDENCE

Use a failure probe to expose the remaining uncertainty

The fictional score record continues the same notification scenario. Scores cite decisions; an unresolved external guarantee remains a named question rather than a confident claim.

FICTIONAL WORKED EXAMPLE - ADAPT THE METHOD, NOT THE FACTS

Dimension	Score + evidence	Quote, timestamp, diagram, or decision evidence
Requirements / non-goals	2	03:00: accepted work must survive; immediate email completion is excluded from the response promise.
Scale / capacity	2	07:00: 5,000 events/s x two recipients = 10,000 deliveries/s before channel fan-out.
Decision order / data flow	2	12:00: durable acceptance and delivery state precede component detail.
Architecture / bottleneck	1	24:00: provider quota identified late after the worker count was chosen.
Trade-off clarity	2	30:00: asynchronous delivery reduces request coupling but requires visible eventual status.
Failure / recovery / metrics	1	28:00: oldest-event age named; crash-after-send outcome still needs provider verification.
Communication / recovery	2	31:00: labels the unresolved guarantee and updates only retry behavior during the pivot.

Worked pivot

Requirement change	What changes	What stays stable
Delivery may be delayed, but duplicate actions are unacceptable	Idempotency key, deduplication store, retry policy, and success metric become explicit	The product contract, source of truth, and unrelated read path remain unchanged

NEUTRAL OBSERVER PROMPTS

Use: What assumption drives that? What fails first? Which decision changes if the requirement changes? Avoid suggesting the component or answer.

FINISH BOTH TASKS

Leave with artifacts and one next correction

The rehearsal is complete when both runs are saved, evidence is scored, and the next test is smaller than the practice block you just finished.

Roadmap task	Required saved artifact	Completion proof
Timed coding rehearsal	Prompt, approach, invariant, complexity, test list, first missed edge	One quote/timestamp and a rubric score tied to code or tests
Timed system-design rehearsal	Prompt, three assumptions and caused decisions, diagram, trade-off to revisit	One quote/timestamp and a rubric score tied to the decision ledger

Common mistakes

+ Starting implementation or architecture before clarifying the contract.
+ Optimizing for speed while dropping tests, failure handling, or a clear close.
+ Using a familiar prompt whose solution is already memorized.
+ Scoring from confidence instead of quotes, artifacts, and decision moments.
+ Choosing several corrections and losing evidence about what worked.

YOUR NEXT ACTION

Choose one lowest evidence dimension, write one correction sentence, and schedule one comparable rehearsal. Keep the prompt shape and rubric stable enough to measure change.

Completion check

- [ ] Both prompts and conditions are recorded.
- [ ] Both clocks preserve the required decision stages.
- [ ] Coding and design artifacts are saved.
- [ ] Scores cite observable evidence.
- [ ] Only one correction is active for the next test.

APPLY THE METHOD TO YOUR OWN EVIDENCE

For a fresh prompt: save contract, examples, invariant, approach, rejected alternative, tests, complexity, and first missed decision. For design: save three assumption-to-decision rows, failure timeline, metric, and a reversal condition.